Observer Pattern (Publish-Subscribe)

Theory, Edge Cases & Code Examples

- ✓ Observer Pattern Overview
- ✓ Push vs Pull Observer Models
- ✓ C++ Implementation Patterns
- ✓ Observer Pattern and Concurrency
- ✓ Autonomous Vehicle Example

→ Swipe to learn →

Observer Pattern (Publish-Subscribe)

Theory

📌 1. Observer Pattern Overview (1/2)

Definition: Behavioral design pattern defining one-to-many dependency between objects where changes in one object (subject) automatically notify and update dependent objects (observers).

Core Components:

Component	Role	Responsibility
Subject	Observable object	Maintains list of observers, sends notifications
Observer	Dependent object	Receives notifications, updates itself
Concrete Subject	Specific observable	Stores state, triggers notifications on change
Concrete Observer	Specific dependent	Implements update logic for notifications

Core Guarantee:

Requirement	Implementation	Purpose
Loose coupling	Observers don't know each other	Reduce interdependencies
Dynamic subscription	Attach/detach observers at runtime	Flexible notification lists
Automatic updates	Subject notifies on state change	Maintain consistency
One-to-many	Single subject, multiple observers	Broadcast updates efficiently

Observer Pattern (Publish-Subscribe)

Theory

📌 1. Observer Pattern Overview (2/2)

When to Use Observer Pattern:

Common Use Cases:

```
class SensorFusion {           // ✅ Multiple components need sensor updates
class EventDispatcher {         // ✅ UI elements react to model changes
class StockPriceMonitor {       // ✅ Multiple displays show same data
class LogManager {              // ✅ Multiple loggers receive same events
class CollisionDetector {       // ✅ Multiple systems react to collision events
```

Inappropriate Uses:

```
class SingletonObserver {       // ❌ Only one observer - use direct call
class TightlyCoupled {          // ❌ Observer needs specific subject knowledge
class HighFrequencyUpdates {    // ❌ 10kHz+ updates - overhead too high
```

Observer Pattern (Publish-Subscribe)

Theory

📌 2. Push vs Pull Observer Models (1/2)

Push Model: Subject sends detailed data with notification

```
// Push model - subject sends all data
class Subject {
    vector<Observer*> observers;
    double sensorValue;

public:
    void notifyObservers() {
        for (auto* obs : observers) {
            obs->update(sensorValue);  // Push data to observer
        }
    }
};

class Observer {
public:
    virtual void update(double value) = 0;  // Receives pushed data
};
```

Pull Model: Observer queries subject for needed data

```
// Pull model - observer queries subject
class Subject {
    vector<Observer*> observers;
    double sensorValue;

public:
    void notifyObservers() {
        for (auto* obs : observers) {
            obs->update(this);  // Pass subject reference
        }
    }

    double getValue() const { return sensorValue; }  // Observers pull data
};

class Observer {
public:
// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Theory

📌 2. Push vs Pull Observer Models (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    virtual void update(Subject* subject) = 0;  // Observer pulls what it needs
};
```

Comparison:

Aspect	Push Model	Pull Model
Data transfer	Subject sends all data	Observer queries what it needs
Coupling	Higher (observers know data format)	Lower (observers query interface)
Efficiency	Wastes bandwidth if observer doesn't need all data	Efficient - observers get only what they need
Simplicity	Simpler observer implementation	More flexible, more complex
Use case	Small, fixed data set	Large or variable data set

Observer Pattern (Publish-Subscribe)

Theory

📌 3. C++ Implementation Patterns (1/4)

A. Raw Pointer Observers (Simple but Dangerous):

```
class Subject {
    vector<Observer*> observers;  // ❌ Raw pointers - lifetime issues

public:
    void attach(Observer* obs) { observers.push_back(obs); }

    void detach(Observer* obs) {
        observers.erase(remove(observers.begin(), observers.end(), obs), observers.end());
    }

    void notify() {
        for (auto* obs : observers) {
            obs->update();  // ❌ Dangling pointer if observer deleted!
        }
    }
};
```

Problem: If observer is deleted without detaching, subject holds dangling pointer.

Observer Pattern (Publish-Subscribe)

Theory

📌 3. C++ Implementation Patterns (2/4)

B. Smart Pointer Observers (Safer):

```
class Subject {
    vector<weak_ptr<Observer>> observers;  // ✅ weak_ptr avoids ownership cycles

public:
    void attach(shared_ptr<Observer> obs) {
        observers.push_back(obs);
    }

    void notify() {
        // Clean up expired observers while iterating
        observers.erase(
            remove_if(observers.begin(), observers.end(),
                      [](const weak_ptr<Observer>& wp) { return wp.expired(); }),
            observers.end()
        );

        // Notify alive observers
        for (auto& wp : observers) {
            if (auto obs = wp.lock()) {  // Convert weak_ptr to shared_ptr
                obs->update();
            }
        }
    }
};
```

Benefit: weak_ptr doesn't affect observer lifetime. Observers can be destroyed without manual detach.

Observer Pattern (Publish-Subscribe)

Theory

📌 3. C++ Implementation Patterns (3/4)

C. Signal/Slot (Boost.Signals2 style):

```
#include <boost/signals2.hpp>

class Subject {
    boost::signals2::signal<void(double)> valueChanged;  // Signal

public:
    void setValue(double val) {
        value = val;
        valueChanged(val);  // Emit signal
    }

    boost::signals2::connection connect(function<void(double)> slot) {
        return valueChanged.connect(slot);  // Connect slot
    }
};

// Usage
Subject subject;
auto connection = subject.connect([](double val) {
    cout << "Value changed: " << val << "\n";
});
connection.disconnect();  // Explicit disconnect
```

Observer Pattern (Publish-Subscribe)

Theory

📌 3. C++ Implementation Patterns (4/4)

D. std::function Callbacks:

```
class Subject {
    vector<function<void(double)>> callbacks;

public:
    void subscribe(function<void(double)> callback) {
        callbacks.push_back(callback);
    }

    void notify(double value) {
        for (auto& cb : callbacks) {
            cb(value);
        }
    }
};

// Usage
Subject subject;
subject.subscribe([](double val) {
    cout << "Callback: " << val << "\n";
});
```

Observer Pattern (Publish-Subscribe)

Theory

📌 4. Observer Pattern and Concurrency (1/2)

Thread Safety Challenges:

Challenge	Problem	Solution
Concurrent attach/detach	Race condition modifying observer list	Mutex protecting observer list
Observer deletion during notification	Iterator invalidation	Copy observer list before notifying
Concurrent notifications	Multiple threads notifying simultaneously	Atomic state + mutex for observer list
Deadlock	Observer callback acquires lock subject needs	Avoid nested locking, use lock-free notifications

Observer Pattern (Publish-Subscribe)

Theory

📌 4. Observer Pattern and Concurrency (2/2)

Thread-Safe Observer (Mutex-Protected):

```
class ThreadSafeSubject {
    mutable mutex mtx;
    vector<weak_ptr<Observer>> observers;

public:
    void attach(shared_ptr<Observer> obs) {
        lock_guard<mutex> lock(mtx);
        observers.push_back(obs);
    }

    void notify() {
        // Copy observers under lock to avoid holding lock during callbacks
        vector<shared_ptr<Observer>> observersCopy;
        {
            lock_guard<mutex> lock(mtx);
            for (auto& wp : observers) {
                if (auto obs = wp.lock()) {
                    observersCopy.push_back(obs);
                }
            }
        }

        // Notify without holding lock
        for (auto& obs : observersCopy) {
            obs->update();
        }
    }
};
```

Observer Pattern (Publish-Subscribe)

Theory

📌 5. Autonomous Vehicle Example (1/2)

Real-World Observer Use Case:

```
// Subject: Collision detection system
class CollisionDetector {
    vector<weak_ptr<CollisionObserver>> observers;
    mutable mutex mtx;

public:
    void attach(shared_ptr<CollisionObserver> obs) {
        lock_guard lock(mtx);
        observers.push_back(obs);
    }

    void detectCollisionRisk(double distance, double closing_speed) {
        if (distance < 5.0 && closing_speed > 2.0) {
            // Collision imminent!
            CollisionEvent event{distance, closing_speed, "CRITICAL"};
            notifyObservers(event);
        }
    }

private:
    void notifyObservers(const CollisionEvent& event) {
        vector<shared_ptr<CollisionObserver>> active;
        {
            lock_guard lock(mtx);
            for (auto& wp : observers) {
                if (auto obs = wp.lock()) active.push_back(obs);
            }
        }

        for (auto& obs : active) {
            obs->onCollisionDetected(event);
        }
    }
};

// Observers: Various vehicle systems
class EmergencyBraking : public CollisionObserver {
public:
    void onCollisionDetected(const CollisionEvent& event) override {
// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Theory

📌 5. Autonomous Vehicle Example (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        if (event.severity == "CRITICAL") {
            cout << "⚠️  EMERGENCY BRAKING ENGAGED!\n";
            applyMaxBraking();
        }
    }
};

class AlertSystem : public CollisionObserver {
public:
    void onCollisionDetected(const CollisionEvent& event) override {
        cout << "🔔 COLLISION WARNING: " << event.distance << "m\n";
        playWarningSound();
    }
};

class DataLogger : public CollisionObserver {
public:
    void onCollisionDetected(const CollisionEvent& event) override {
        logToFile("Collision event: " + event.toString());
    }
};

// Setup
auto detector = make_shared<CollisionDetector>();
auto braking = make_shared<EmergencyBraking>();
auto alert = make_shared<AlertSystem>();
auto logger = make_shared<DataLogger>();

detector->attach(braking);
detector->attach(alert);
detector->attach(logger);

// When collision detected, all three systems automatically respond
detector->detectCollisionRisk(3.5, 5.0);  // Notifies all observers
```

Observer Pattern (Publish-Subscribe)

Theory

📌 6. Why Observer Pattern Matters

Critical Concepts Demonstrated:

Concept	How Observer Tests It	Interview Relevance
Polymorphism	Observers accessed through abstract interface	Core OOP understanding
Loose coupling	Subject doesn't know concrete observer types	Design for maintainability
Lifetime management	Smart pointers prevent dangling references	Modern C++ resource safety
Concurrency	Thread-safe notification requires careful design	Production system requirements
Design patterns	Classic GoF behavioral pattern	Software architecture knowledge

Common Interview Questions:

- "What's the difference between Observer and Publish-Subscribe?" (Observer: direct reference; Pub-Sub: event channel mediator)
- "How do you prevent memory leaks in Observer pattern?" (Use weak_ptr or explicit detach)
- "What's the difference between push and pull observer models?" (Push sends data; pull provides access)
- "How do you make Observer thread-safe?" (Mutex + copy observer list before notifying)

Observer Pattern (Publish-Subscribe)

Edge Cases

📌 1. Circular References and Memory Leaks (1/2)

```
// ❌ PROBLEM: shared_ptr circular reference causes memory leak
class Subject {
    vector<shared_ptr<Observer>> observers;  // ❌ Strong reference

public:
    void attach(shared_ptr<Observer> obs) {
        observers.push_back(obs);  // Subject owns observer
    }
};

class Observer {
    shared_ptr<Subject> subject;  // ❌ Observer owns subject

public:
    Observer(shared_ptr<Subject> subj) : subject(subj) {
        subject->attach(shared_from_this());  // Circular reference!
    }
};

// Neither Subject nor Observer ever destroyed - memory leak!

// ✅ SOLUTION: Use weak_ptr to break cycle
class Subject {
    vector<weak_ptr<Observer>> observers;  // ✅ Weak reference

public:
    void attach(shared_ptr<Observer> obs) {
        observers.push_back(obs);
    }

    void notify() {
        for (auto& wp : observers) {
            if (auto obs = wp.lock()) {  // Check if still alive
                obs->update();
            }
        }
    }
};

class Observer : public enable_shared_from_this<Observer> {
// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Edge Cases

📌 1. Circular References and Memory Leaks (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    weak_ptr<Subject> subject;  // ✅ Weak reference to subject

public:
    Observer(shared_ptr<Subject> subj) : subject(subj) {
        if (auto s = subject.lock()) {
            s->attach(shared_from_this());
        }
    }
};
```

Why This Matters: Observer pattern naturally creates circular references (subject knows observers, observers know subject). Always use `weak_ptr` for at least one direction to prevent memory leaks.

Observer Pattern (Publish-Subscribe)

Edge Cases

📌 2. Observer Deletion During Notification (1/2)

```
// ❌ PROBLEM: Observer removes itself during notification loop
class Subject {
    vector<Observer*> observers;

public:
    void notify() {
        for (auto* obs : observers) {  // ❌ Iterating observers
            obs->update();  // Observer might call detach() here!
        }
    }

    void detach(Observer* obs) {
        // ❌ Modifies vector while being iterated - undefined behavior!
        observers.erase(remove(observers.begin(), observers.end(), obs), observers.end());
    }
};

class SelfRemovingObserver : public Observer {
    Subject* subject;

public:
    void update() override {
        subject->detach(this);  // ❌ Removes self during iteration!
    }
};

// ✅ SOLUTION: Copy observer list before notifying
class Subject {
    vector<weak_ptr<Observer>> observers;
    mutable mutex mtx;

public:
    void notify() {
        // Copy observers to avoid iteration during modification
        vector<shared_ptr<Observer>> observersCopy;
        {
            lock_guard lock(mtx);
            for (auto& wp : observers) {
                if (auto obs = wp.lock()) {
                    observersCopy.push_back(obs);
// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Edge Cases

📌 2. Observer Deletion During Notification (2/2)

[Code continues across slides]

```
// ...continued from previous slide
                }
            }
        }

        // Notify on copy - safe if observers detach themselves
        for (auto& obs : observersCopy) {
            obs->update();
        }
    }
};
```

Why This Matters: Observers often unsubscribe in response to notifications (e.g., one-time listeners). Copying the observer list before iterating prevents iterator invalidation.

Observer Pattern (Publish-Subscribe)

Edge Cases

📌 3. Notification Storms and Infinite Loops (1/2)

```
// ❌ PROBLEM: Observer triggers notification that triggers observer again
class TemperatureSubject {
    double temperature;
    vector<weak_ptr<Observer>> observers;

public:
    void setTemperature(double temp) {
        temperature = temp;
        notify();  // Notifies observers
    }
};

class TemperatureController : public Observer {
    shared_ptr<TemperatureSubject> subject;

public:
    void update() override {
        double current = subject->getTemperature();
        if (current < 20.0) {
            // ❌ Setting temperature triggers notification again!
            subject->setTemperature(current + 1.0);  // Infinite loop!
        }
    }
};

// ✅ SOLUTION 1: Guard flag to prevent recursive notifications
class TemperatureSubject {
    double temperature;
    bool notifying = false;  // Recursion guard
    vector<weak_ptr<Observer>> observers;

public:
    void setTemperature(double temp) {
        if (temperature != temp) {  // Only notify on actual change
            temperature = temp;

            if (!notifying) {  // ✅ Prevent recursive notifications
                notifying = true;
                notify();
                notifying = false;
// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Edge Cases

📌 3. Notification Storms and Infinite Loops (2/2)

[Code continues across slides]

```
// ...continued from previous slide
            }
        }
    }
};

// ✅ SOLUTION 2: Deferred notifications (queue updates)
class TemperatureSubject {
    double temperature;
    queue<double> pendingUpdates;
    bool processing = false;

public:
    void setTemperature(double temp) {
        pendingUpdates.push(temp);

        if (!processing) {
            processing = true;
            while (!pendingUpdates.empty()) {
                temperature = pendingUpdates.front();
                pendingUpdates.pop();
                notify();
            }
            processing = false;
        }
    }
};
```

Why This Matters: Observers that modify the subject can create notification storms or infinite loops. Use recursion guards or deferred notifications to prevent cascading updates.

Observer Pattern (Publish-Subscribe)

Edge Cases

📌 4. Thread Safety and Race Conditions (1/2)

```
// ❌ PROBLEM: Concurrent attach/notify creates race conditions
class Subject {
    vector<weak_ptr<Observer>> observers;  // ❌ Not protected

public:
    void attach(shared_ptr<Observer> obs) {
        observers.push_back(obs);  // ❌ Race condition with notify()
    }

    void notify() {
        for (auto& wp : observers) {  // ❌ Concurrent modification
            if (auto obs = wp.lock()) {
                obs->update();
            }
        }
    }
};

// Thread 1: subject.attach(obs1);
// Thread 2: subject.notify();  // ❌ Data race on observers vector!

// ✅ SOLUTION: Mutex protection with careful locking
class ThreadSafeSubject {
    mutable mutex mtx;
    vector<weak_ptr<Observer>> observers;

public:
    void attach(shared_ptr<Observer> obs) {
        lock_guard lock(mtx);  // ✅ Protect modification
        observers.push_back(obs);
    }

    void notify() {
        // Step 1: Copy observers under lock
        vector<shared_ptr<Observer>> snapshot;
        {
            lock_guard lock(mtx);
            for (auto& wp : observers) {
                if (auto obs = wp.lock()) {
                    snapshot.push_back(obs);
// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Edge Cases

📌 4. Thread Safety and Race Conditions (2/2)

[Code continues across slides]

```
// ...continued from previous slide
                }
            }
        }  // Release lock before callbacks

        // Step 2: Notify without holding lock (avoid deadlock)
        for (auto& obs : snapshot) {
            obs->update();
        }
    }
};
```

Why This Matters: Multi-threaded systems commonly use Observer pattern. Without proper synchronization, concurrent attach/detach/notify operations cause data races and undefined behavior. Always protect observer list with mutex, and release lock before invoking callbacks to avoid deadlocks.

Observer Pattern (Publish-Subscribe)

Edge Cases

📌 5. Observer Order and Dependencies (1/3)

```
// ❌ PROBLEM: Observers have implicit dependencies on execution order
class SensorSubject {
    vector<weak_ptr<Observer>> observers;

public:
    void notify() {
        for (auto& wp : observers) {  // ❌ Order undefined
            if (auto obs = wp.lock()) {
                obs->update();
            }
        }
    }
};

// Observer A must execute before Observer B
class DataValidator : public Observer {
    void update() override {
        validateData();  // Must run first
    }
};

class DataProcessor : public Observer {
    void update() override {
        processData();  // ❌ Assumes data already validated!
    }
};

// If DataProcessor notified before DataValidator, processes invalid data!

// ✅ SOLUTION 1: Priority-based notification
class Subject {
    struct ObserverEntry {
        weak_ptr<Observer> observer;
        int priority;  // Higher = notified first

        bool operator<(const ObserverEntry& other) const {
            return priority > other.priority;  // Descending order
        }
    };

// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Edge Cases

📌 5. Observer Order and Dependencies (2/3)

[Code continues across slides]

```
// ...continued from previous slide
    vector<ObserverEntry> observers;

public:
    void attach(shared_ptr<Observer> obs, int priority = 0) {
        observers.push_back({obs, priority});
        sort(observers.begin(), observers.end());  // Sort by priority
    }

    void notify() {
        for (auto& entry : observers) {
            if (auto obs = entry.observer.lock()) {
                obs->update();
            }
        }
    }
};

// Usage
subject.attach(validator, 100);   // High priority
subject.attach(processor, 50);    // Lower priority - runs after validator

// ✅ SOLUTION 2: Multi-phase notification
class Subject {
public:
    void notify() {
        // Phase 1: Validators
        for (auto& obs : validatorObservers) {
            obs->update();
        }

        // Phase 2: Processors (run after validation)
        for (auto& obs : processorObservers) {
            obs->update();
        }

        // Phase 3: Loggers (run last)
        for (auto& obs : loggerObservers) {
            obs->update();
        }
    }
```

Observer Pattern (Publish-Subscribe)

Edge Cases

📌 5. Observer Order and Dependencies (3/3)

Why This Matters: Observer pattern doesn't guarantee notification order. If observers have dependencies, use explicit priorities or multi-phase notifications to ensure correct execution order.

Observer Pattern (Publish-Subscribe)

Edge Cases

📌 6. Exception Safety in Observer Callbacks (1/2)

```
// ❌ PROBLEM: Observer throws exception, remaining observers not notified
class Subject {
    vector<weak_ptr<Observer>> observers;

public:
    void notify() {
        for (auto& wp : observers) {
            if (auto obs = wp.lock()) {
                obs->update();  // ❌ If throws, loop aborts!
            }
        }
    }
};

class ThrowingObserver : public Observer {
    void update() override {
        throw runtime_error("Update failed!");  // ❌ Prevents other observers from running
    }
};

// ✅ SOLUTION: Catch exceptions, continue notifying
class Subject {
    vector<weak_ptr<Observer>> observers;

public:
    void notify() {
        for (auto& wp : observers) {
            if (auto obs = wp.lock()) {
                try {
                    obs->update();  // ✅ Isolated from exceptions
                } catch (const exception& e) {
                    cerr << "Observer update failed: " << e.what() << "\n";
                    // Log error but continue notifying other observers
                }
            }
        }
    }
};

// ✅ ALTERNATIVE: Collect exceptions and throw aggregate
// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Edge Cases

📌 6. Exception Safety in Observer Callbacks (2/2)

[Code continues across slides]

```
// ...continued from previous slide
class Subject {
public:
    void notify() {
        vector<exception_ptr> exceptions;

        for (auto& wp : observers) {
            if (auto obs = wp.lock()) {
                try {
                    obs->update();
                } catch (...) {
                    exceptions.push_back(current_exception());
                }
            }
        }

        if (!exceptions.empty()) {
            // Throw first exception or custom aggregate exception
            rethrow_exception(exceptions[0]);
        }
    }
};
```

Why This Matters: Observer callbacks are user code - they can throw exceptions. Without exception handling, one failing observer prevents others from being notified. Always catch exceptions during notification to ensure all observers execute.

Observer Pattern (Publish-Subscribe)

Code Examples

📌 1. Easy - Basic Observer with Raw Pointers (1/3)

Basic Observer pattern with raw pointers. Subject maintains list of observers and notifies them when state changes. Simple but requires manual subscription management.

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <string>

// Observer interface
class Observer {
public:
    virtual ~Observer() = default;
    virtual void update(const std::string& message) = 0;
};

// Subject (Observable)
class NewsAgency {
    std::vector<Observer*> observers;
    std::string latestNews;

public:
    void subscribe(Observer* observer) {
        observers.push_back(observer);
    }

    void unsubscribe(Observer* observer) {
        observers.erase(
            std::remove(observers.begin(), observers.end(), observer),
            observers.end()
        );
    }

    void setNews(const std::string& news) {
        latestNews = news;
        notifyObservers();
    }

private:
    void notifyObservers() {
// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Code Examples

📌 1. Easy - Basic Observer with Raw Pointers (2/3)

```
// ...continued from previous slide
        for (auto* observer : observers) {
            observer->update(latestNews);
        }
    }
};

// Concrete Observers
class EmailSubscriber : public Observer {
    std::string email;

public:
    EmailSubscriber(const std::string& e) : email(e) {}

    void update(const std::string& message) override {
        std::cout << "Email to " << email << ": " << message << "\n";
    }
};

class SMSSubscriber : public Observer {
    std::string phoneNumber;

public:
    SMSSubscriber(const std::string& phone) : phoneNumber(phone) {}

    void update(const std::string& message) override {
        std::cout << "SMS to " << phoneNumber << ": " << message << "\n";
    }
};

int main() {
    std::cout << "Basic Observer Pattern Example\n\n";

    NewsAgency agency;

    EmailSubscriber email1("user@example.com");
    SMSSubscriber sms1("555-1234");

    // Subscribe observers
    agency.subscribe(&email1);
    agency.subscribe(&sms1);
// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Code Examples

📌 1. Easy - Basic Observer with Raw Pointers (3/3)

[Code continues across slides]

```
// ...continued from previous slide

    // Publish news - all subscribers notified
    agency.setNews("Breaking: C++ Observer Pattern Explained!");

    std::cout << "\nUnsubscribing email...\n";
    agency.unsubscribe(&email1);

    agency.setNews("Update: Only SMS subscriber receives this");

    return 0;
}
```

Observer Pattern (Publish-Subscribe)

Code Examples

📌 2. Mid - Smart Pointer Observer (weak_ptr) (1/3)

Smart pointer observer using `weak_ptr` to avoid memory leaks. Subject doesn't own observers, so they can be destroyed without manual detachment. Automatic cleanup of dead observers.

```
#include <iostream>
#include <vector>
#include <memory>
#include <algorithm>
#include <string>

// Observer interface
class Observer {
public:
    virtual ~Observer() = default;
    virtual void update(double temperature) = 0;
};

// Subject using smart pointers
class TemperatureSensor {
    std::vector<std::weak_ptr<Observer>> observers;
    double currentTemperature = 20.0;

public:
    void attach(std::shared_ptr<Observer> observer) {
        observers.push_back(observer);  // Store weak_ptr
    }

    void setTemperature(double temp) {
        currentTemperature = temp;
        notifyObservers();
    }

    double getTemperature() const {
        return currentTemperature;
    }

private:
    void notifyObservers() {
        // Clean up expired observers
        observers.erase(
// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Code Examples

📌 2. Mid - Smart Pointer Observer (weak_ptr) (2/3)

```
// ...continued from previous slide
            std::remove_if(observers.begin(), observers.end(),
                           [](const std::weak_ptr<Observer>& wp) {
                               return wp.expired();
                           }),
            observers.end()
        );

        // Notify alive observers
        for (auto& wp : observers) {
            if (auto observer = wp.lock()) {  // Convert to shared_ptr
                observer->update(currentTemperature);
            }
        }
    }
};

// Concrete Observers
class TemperatureDisplay : public Observer {
    std::string location;

public:
    TemperatureDisplay(const std::string& loc) : location(loc) {}

    ~TemperatureDisplay() {
        std::cout << "Display at " << location << " destroyed\n";
    }

    void update(double temperature) override {
        std::cout << "[" << location << "] Temperature: "
                  << temperature << "°C\n";
    }
};

class TemperatureAlert : public Observer {
    double threshold;

public:
    TemperatureAlert(double t) : threshold(t) {}

    void update(double temperature) override {
// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Code Examples

📌 2. Mid - Smart Pointer Observer (weak_ptr) (3/3)

[Code continues across slides]

```
// ...continued from previous slide
        if (temperature > threshold) {
            std::cout << "⚠️  ALERT: Temperature " << temperature
                      << "°C exceeds threshold " << threshold << "°C!\n";
        }
    }
};

int main() {
    std::cout << "Smart Pointer Observer Example\n\n";

    TemperatureSensor sensor;

    auto display1 = std::make_shared<TemperatureDisplay>("Living Room");
    auto display2 = std::make_shared<TemperatureDisplay>("Bedroom");
    auto alert = std::make_shared<TemperatureAlert>(25.0);

    sensor.attach(display1);
    sensor.attach(display2);
    sensor.attach(alert);

    std::cout << "Setting temperature to 22°C:\n";
    sensor.setTemperature(22.0);

    std::cout << "\nSetting temperature to 28°C:\n";
    sensor.setTemperature(28.0);

    std::cout << "\nDestroying one display...\n";
    display1.reset();  // Destroy observer

    std::cout << "\nSetting temperature to 30°C:\n";
    sensor.setTemperature(30.0);  // Only 2 observers notified

    return 0;
}
```

Observer Pattern (Publish-Subscribe)

Code Examples

📌 3. Mid - Pull Model Observer (1/3)

Pull model observer where observers query subject for needed data. More flexible than push model - each observer gets only what it needs. Lower coupling.

```
#include <iostream>
#include <vector>
#include <memory>
#include <string>

// Forward declaration
class WeatherStation;

// Observer interface (Pull model)
class WeatherObserver {
public:
    virtual ~WeatherObserver() = default;
    virtual void update(WeatherStation* station) = 0;  // Pull data from station
};

// Subject with multiple data points
class WeatherStation {
    std::vector<std::weak_ptr<WeatherObserver>> observers;
    double temperature = 20.0;
    double humidity = 50.0;
    double pressure = 1013.0;

public:
    void attach(std::shared_ptr<WeatherObserver> obs) {
        observers.push_back(obs);
    }

    void setMeasurements(double temp, double hum, double pres) {
        temperature = temp;
        humidity = hum;
        pressure = pres;
        notifyObservers();
    }

    // Pull interface - observers query what they need
    double getTemperature() const { return temperature; }
// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Code Examples

📌 3. Mid - Pull Model Observer (2/3)

```
// ...continued from previous slide
    double getHumidity() const { return humidity; }
    double getPressure() const { return pressure; }

private:
    void notifyObservers() {
        for (auto& wp : observers) {
            if (auto obs = wp.lock()) {
                obs->update(this);  // Pass station reference
            }
        }
    }
};

// Concrete Observers (each pulls different data)
class TemperatureDisplay : public WeatherObserver {
public:
    void update(WeatherStation* station) override {
        // Only pulls temperature
        double temp = station->getTemperature();
        std::cout << "Temperature Display: " << temp << "°C\n";
    }
};

class HumidityDisplay : public WeatherObserver {
public:
    void update(WeatherStation* station) override {
        // Only pulls humidity
        double hum = station->getHumidity();
        std::cout << "Humidity Display: " << hum << "%\n";
    }
};

class FullWeatherDisplay : public WeatherObserver {
public:
    void update(WeatherStation* station) override {
        // Pulls all data
        std::cout << "Full Weather Report:\n";
        std::cout << "  Temperature: " << station->getTemperature() << "°C\n";
        std::cout << "  Humidity: " << station->getHumidity() << "%\n";
        std::cout << "  Pressure: " << station->getPressure() << " hPa\n";
// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Code Examples

📌 3. Mid - Pull Model Observer (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    }
};

int main() {
    std::cout << "Pull Model Observer Example\n\n";

    WeatherStation station;

    auto tempDisplay = std::make_shared<TemperatureDisplay>();
    auto humDisplay = std::make_shared<HumidityDisplay>();
    auto fullDisplay = std::make_shared<FullWeatherDisplay>();

    station.attach(tempDisplay);
    station.attach(humDisplay);
    station.attach(fullDisplay);

    std::cout << "Weather update 1:\n";
    station.setMeasurements(22.5, 60.0, 1015.0);

    std::cout << "\nWeather update 2:\n";
    station.setMeasurements(25.0, 55.0, 1012.0);

    return 0;
}
```

Observer Pattern (Publish-Subscribe)

Code Examples

📌 4. Advanced - Thread-Safe Observer (1/3)

Thread-safe observer with mutex protection. Copies observer list before notifying to avoid holding lock during callbacks (prevents deadlock). Safe for concurrent attach/notify operations.

```
#include <iostream>
#include <vector>
#include <memory>
#include <mutex>
#include <thread>
#include <chrono>

// Observer interface
class SensorObserver {
public:
    virtual ~SensorObserver() = default;
    virtual void onSensorUpdate(double value) = 0;
};

// Thread-safe Subject
class ThreadSafeSensor {
    mutable std::mutex mtx;
    std::vector<std::weak_ptr<SensorObserver>> observers;
    double sensorValue = 0.0;

public:
    void attach(std::shared_ptr<SensorObserver> obs) {
        std::lock_guard<std::mutex> lock(mtx);
        observers.push_back(obs);
    }

    void setSensorValue(double value) {
        {
            std::lock_guard<std::mutex> lock(mtx);
            sensorValue = value;
        }
        notifyObservers(value);
    }

private:
    void notifyObservers(double value) {
// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Code Examples

📌 4. Advanced - Thread-Safe Observer (2/3)

```
// ...continued from previous slide
        // Copy observers under lock
        std::vector<std::shared_ptr<SensorObserver>> snapshot;
        {
            std::lock_guard<std::mutex> lock(mtx);

            // Clean expired and copy alive observers
            observers.erase(
                std::remove_if(observers.begin(), observers.end(),
                               [](const auto& wp) { return wp.expired(); }),
                observers.end()
            );

            for (auto& wp : observers) {
                if (auto obs = wp.lock()) {
                    snapshot.push_back(obs);
                }
            }
        }  // Release lock before callbacks

        // Notify without holding lock (avoid deadlock)
        for (auto& obs : snapshot) {
            obs->onSensorUpdate(value);
        }
    }
};

// Concrete Observer
class SensorLogger : public SensorObserver {
    std::string name;
    mutable std::mutex logMtx;

public:
    SensorLogger(const std::string& n) : name(n) {}

    void onSensorUpdate(double value) override {
        std::lock_guard<std::mutex> lock(logMtx);
        std::cout << "[" << name << "] Sensor value: " << value << "\n";
    }
};

// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Code Examples

📌 4. Advanced - Thread-Safe Observer (3/3)

[Code continues across slides]

```
// ...continued from previous slide
int main() {
    std::cout << "Thread-Safe Observer Example\n\n";

    ThreadSafeSensor sensor;

    auto logger1 = std::make_shared<SensorLogger>("Logger-1");
    auto logger2 = std::make_shared<SensorLogger>("Logger-2");

    sensor.attach(logger1);
    sensor.attach(logger2);

    // Simulate concurrent sensor updates from multiple threads
    std::vector<std::thread> threads;

    for (int i = 0; i < 5; ++i) {
        threads.emplace_back([&sensor, i]() {
            for (int j = 0; j < 3; ++j) {
                sensor.setSensorValue(i * 10 + j);
                std::this_thread::sleep_for(std::chrono::milliseconds(10));
            }
        });
    }

    for (auto& t : threads) {
        t.join();
    }

    std::cout << "\nAll threads completed\n";

    return 0;
}
```

Observer Pattern (Publish-Subscribe)

Code Examples

📌 5. Advanced - Signal/Slot with std::function (1/3)

Signal/Slot pattern using `std::function`. Type-safe, flexible callback system. Each signal can have multiple slots (callbacks). Supports disconnection by ID.

```
#include <iostream>
#include <vector>
#include <functional>
#include <memory>
#include <algorithm>

// Signal class (Observable)
template<typename... Args>
class Signal {
    using Slot = std::function<void(Args...)>;

    struct Connection {
        int id;
        Slot slot;
    };

    std::vector<Connection> connections;
    int nextId = 0;

public:
    // Connect a slot (returns connection ID)
    int connect(Slot slot) {
        int id = nextId++;
        connections.push_back({id, slot});
        return id;
    }

    // Disconnect by ID
    void disconnect(int connectionId) {
        connections.erase(
            std::remove_if(connections.begin(), connections.end(),
                           [connectionId](const Connection& conn) {
                               return conn.id == connectionId;
                           }),
            connections.end()
        );
// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Code Examples

📌 5. Advanced - Signal/Slot with std::function (2/3)

```
// ...continued from previous slide
    }

    // Emit signal - notify all connected slots
    void emit(Args... args) {
        for (auto& conn : connections) {
            conn.slot(args...);
        }
    }

    // Operator overload for easier emission
    void operator()(Args... args) {
        emit(args...);
    }
};

// Example: Button class with clicked signal
class Button {
    std::string label;

public:
    Signal<const std::string&> clicked;  // Signal with string parameter

    Button(const std::string& lbl) : label(lbl) {}

    void click() {
        std::cout << "Button '" << label << "' clicked!\n";
        clicked(label);  // Emit signal
    }
};

int main() {
    std::cout << "Signal/Slot Pattern Example\n\n";

    Button button("Submit");

    // Connect lambda slots
    auto conn1 = button.clicked.connect([](const std::string& label) {
        std::cout << "  Handler 1: Button " << label << " was clicked\n";
    });

// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Code Examples

📌 5. Advanced - Signal/Slot with std::function (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    auto conn2 = button.clicked.connect([](const std::string& label) {
        std::cout << "  Handler 2: Logging click on " << label << "\n";
    });

    auto conn3 = button.clicked.connect([](const std::string& label) {
        std::cout << "  Handler 3: Sending analytics for " << label << "\n";
    });

    std::cout << "First click:\n";
    button.click();

    std::cout << "\nDisconnecting handler 2...\n";
    button.clicked.disconnect(conn2);

    std::cout << "\nSecond click:\n";
    button.click();

    return 0;
}
```

Observer Pattern (Publish-Subscribe)

Code Examples

📌 7. Advanced - Observer with Priority and Filtering (1/4)

Advanced observer with priority-based notification and event filtering. Observers declare interest in specific event types. High-priority observers (alerts) run before low-priority ones (loggers).

```
#include <iostream>
#include <vector>
#include <memory>
#include <algorithm>
#include <functional>

// Event types
enum class EventType { TEMPERATURE, PRESSURE, HUMIDITY };

struct SensorEvent {
    EventType type;
    double value;
};

// Observer with filtering capability
class FilteredObserver {
public:
    virtual ~FilteredObserver() = default;
    virtual void onEvent(const SensorEvent& event) = 0;
    virtual bool interestedIn(EventType type) const = 0;
};

// Subject with priority and filtering
class SmartSensorSubject {
    struct ObserverEntry {
        std::weak_ptr<FilteredObserver> observer;
        int priority;  // Higher = notified first

        bool operator<(const ObserverEntry& other) const {
            return priority > other.priority;  // Descending
        }
    };

    std::vector<ObserverEntry> observers;

public:
// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Code Examples

📌 7. Advanced - Observer with Priority and Filtering (2/4)

```
// ...continued from previous slide
    void attach(std::shared_ptr<FilteredObserver> obs, int priority = 0) {
        observers.push_back({obs, priority});
        std::sort(observers.begin(), observers.end());
    }

    void publishEvent(const SensorEvent& event) {
        std::cout << "Publishing event: Type="
                  << static_cast<int>(event.type)
                  << ", Value=" << event.value << "\n";

        for (auto& entry : observers) {
            if (auto obs = entry.observer.lock()) {
                if (obs->interestedIn(event.type)) {  // Filter
                    obs->onEvent(event);
                }
            }
        }
    }
};

// Concrete Observers with filters
class TemperatureMonitor : public FilteredObserver {
public:
    bool interestedIn(EventType type) const override {
        return type == EventType::TEMPERATURE;
    }

    void onEvent(const SensorEvent& event) override {
        std::cout << "  [TempMonitor] Temperature: " << event.value << "°C\n";
    }
};

class AllEventsLogger : public FilteredObserver {
public:
    bool interestedIn(EventType type) const override {
        return true;  // Interested in all events
    }

    void onEvent(const SensorEvent& event) override {
        std::cout << "  [Logger] Logging event type "
// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Code Examples

📌 7. Advanced - Observer with Priority and Filtering (3/4)

```
// ...continued from previous slide
                  << static_cast<int>(event.type) << "\n";
    }
};

class CriticalAlerts : public FilteredObserver {
    double tempThreshold = 30.0;
    double pressureThreshold = 1020.0;

public:
    bool interestedIn(EventType type) const override {
        return type == EventType::TEMPERATURE || type == EventType::PRESSURE;
    }

    void onEvent(const SensorEvent& event) override {
        if (event.type == EventType::TEMPERATURE && event.value > tempThreshold) {
            std::cout << "  [ALERT] 🚨 High temperature: " << event.value << "°C!\n";
        }
        if (event.type == EventType::PRESSURE && event.value > pressureThreshold) {
            std::cout << "  [ALERT] 🚨 High pressure: " << event.value << " hPa!\n";
        }
    }
};

int main() {
    std::cout << "Observer with Priority and Filtering\n\n";

    SmartSensorSubject sensor;

    auto tempMonitor = std::make_shared<TemperatureMonitor>();
    auto logger = std::make_shared<AllEventsLogger>();
    auto alerts = std::make_shared<CriticalAlerts>();

    // Attach with priorities (alerts highest, logger lowest)
    sensor.attach(alerts, 100);       // Priority 100 - runs first
    sensor.attach(tempMonitor, 50);   // Priority 50
    sensor.attach(logger, 10);        // Priority 10 - runs last

    std::cout << "Event 1: Temperature 25°C\n";
    sensor.publishEvent({EventType::TEMPERATURE, 25.0});

// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Code Examples

📌 7. Advanced - Observer with Priority and Filtering (4/4)

[Code continues across slides]

```
// ...continued from previous slide
    std::cout << "\nEvent 2: Pressure 1015 hPa\n";
    sensor.publishEvent({EventType::PRESSURE, 1015.0});

    std::cout << "\nEvent 3: Temperature 35°C (critical!)\n";
    sensor.publishEvent({EventType::TEMPERATURE, 35.0});

    std::cout << "\nEvent 4: Humidity 60% (only logger interested)\n";
    sensor.publishEvent({EventType::HUMIDITY, 60.0});

    return 0;
}
```

Observer Pattern (Publish-Subscribe)

Code Examples

📌 8. Performance - Observer with Event Queuing (1/4)

High-performance asynchronous observer with event queue. Events are published to queue and processed by background thread. Prevents blocking caller during observer notifications. Critical for real-time systems.

```
#include <iostream>
#include <vector>
#include <memory>
#include <queue>
#include <mutex>
#include <thread>
#include <chrono>
#include <condition_variable>

// Event structure
struct Event {
    std::string type;
    double value;
    long timestamp;
};

// Observer interface
class AsyncObserver {
public:
    virtual ~AsyncObserver() = default;
    virtual void onEvent(const Event& event) = 0;
};

// Asynchronous observer with event queue
class AsyncEventSubject {
    std::vector<std::weak_ptr<AsyncObserver>> observers;
    std::queue<Event> eventQueue;
    std::mutex mtx;
    std::condition_variable cv;
    bool running = true;
    std::thread workerThread;

public:
    AsyncEventSubject() {
        // Background thread processes events
        workerThread = std::thread([this]() {
// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Code Examples

📌 8. Performance - Observer with Event Queuing (2/4)

```
// ...continued from previous slide
            processEvents();
        });
    }

    ~AsyncEventSubject() {
        {
            std::lock_guard lock(mtx);
            running = false;
        }
        cv.notify_one();
        if (workerThread.joinable()) {
            workerThread.join();
        }
    }

    void attach(std::shared_ptr<AsyncObserver> obs) {
        std::lock_guard lock(mtx);
        observers.push_back(obs);
    }

    void publishEvent(const Event& event) {
        {
            std::lock_guard lock(mtx);
            eventQueue.push(event);
        }
        cv.notify_one();  // Wake up worker thread
    }

private:
    void processEvents() {
        while (true) {
            std::unique_lock lock(mtx);

            // Wait for events or shutdown
            cv.wait(lock, [this]() {
                return !eventQueue.empty() || !running;
            });

            if (!running && eventQueue.empty()) {
                break;  // Shutdown
// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Code Examples

📌 8. Performance - Observer with Event Queuing (3/4)

```
// ...continued from previous slide
            }

            if (!eventQueue.empty()) {
                Event event = eventQueue.front();
                eventQueue.pop();

                // Copy observers
                std::vector<std::shared_ptr<AsyncObserver>> active;
                for (auto& wp : observers) {
                    if (auto obs = wp.lock()) {
                        active.push_back(obs);
                    }
                }

                lock.unlock();  // Release lock before callbacks

                // Notify observers
                for (auto& obs : active) {
                    obs->onEvent(event);
                }
            }
        }
    }
};

// Concrete Observer
class EventProcessor : public AsyncObserver {
    std::string name;

public:
    EventProcessor(const std::string& n) : name(n) {}

    void onEvent(const Event& event) override {
        // Simulate processing time
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
        std::cout << "[" << name << "] Processed event: " << event.type
                  << " = " << event.value << "\n";
    }
};

// ...continues on next slide
```

Observer Pattern (Publish-Subscribe)

Code Examples

📌 8. Performance - Observer with Event Queuing (4/4)

[Code continues across slides]

```
// ...continued from previous slide
int main() {
    std::cout << "Asynchronous Observer with Event Queue\n\n";

    AsyncEventSubject subject;

    auto processor1 = std::make_shared<EventProcessor>("Processor-1");
    auto processor2 = std::make_shared<EventProcessor>("Processor-2");

    subject.attach(processor1);
    subject.attach(processor2);

    std::cout << "Publishing events asynchronously...\n";

    // Publish multiple events quickly
    for (int i = 0; i < 5; ++i) {
        subject.publishEvent({"Temperature", 20.0 + i, 1000 + i});
        std::cout << "Published event " << i << "\n";
    }

    std::cout << "\nWaiting for processing to complete...\n";
    std::this_thread::sleep_for(std::chrono::seconds(2));

    std::cout << "\nAll events processed\n";

    return 0;
}
```

What You Learned

Key Takeaways

- ✓ Observer Pattern Overview
- ✓ Push vs Pull Observer Models
- ✓ C++ Implementation Patterns
- ✓ Observer Pattern and Concurrency

→ *Swipe to learn* →